PROJECT TECHNICAL REPORT

Chatbot Using ChatterBot in Python

An end-to-end guide and system specification for building, training, and deploying a conversational AI chatbot using Python, ChatterBot corpus, and custom YAML datasets.

AUTHOR / DEVELOPER
Rajesh

CATEGORY
Artificial Intelligence / NLP

1. Project Overview & Architecture

This project provides a complete implementation of a command-line interface (CLI) chatbot built using Python and the **ChatterBot** machine learning framework. The chatbot generates automated responses based on input patterns using logic adapters and a SQLite database for persistent memory.

The system is designed with scalability in mind, incorporating corpus training, custom YAML datasets, error handling, clean repository layout, and modular training scripts suitable for GitHub portfolio integration.

> **Key Highlights & Features**
>
> - **Interactive CLI Engine:** Live interactive conversational loop with clean command exit capabilities.
> - **Dual Training Pipeline:** Trained on standard English ChatterBot corpus and custom domain-specific YAML files.
> - **Persistent Storage:** Uses standard SQLite3 database backends to store learned statement logic and response graphs.
> - **Extensible Architecture:** Pre-configured structure ready for REST API wrapper (Flask/FastAPI) or GUI frontend additions.

2. Software & System Requirements

Component	Requirement / Recommendation
Programming Language	Python 3.9 or 3.10 (Recommended for dependency compatibility)
IDE / Editor	VS Code, PyCharm, or Jupyter Notebook
Version Control	Git & GitHub
Core Libraries	`chatterbot`, `chatterbot-corpus`, `spacy`, `pyyaml`
NLP Language Model	`en_core_web_sm` (spaCy English model)

3. Repository & Directory Structure

A well-structured production layout ensures modularity and easy setup for GitHub deployment:

```
Chatbot-Using-ChatterBot/
├── chatbot.py           # Main chatbot execution script
├── train_bot.py        # Dedicated custom model training script
├── requirements.txt    # Project dependencies
├── README.md           # Project documentation and deployment guide
├── LICENSE             # MIT License file
├── .gitignore          # Ignored files (virtual environments, database files)
├── data/
│   └── custom.yml      # Custom conversation training dataset
├── database/
│   └── db.sqlite3      # SQLite database storing statement logic
├── screenshots/
│   ├── output.png      # CLI interaction sample screenshot
│   └── terminal.png
├── docs/
│   ├── Project_Report.pdf
│   └── Presentation.pptx
└── assets/
    ├── flowchart.png
    └── architecture.png
```

4. Step-by-Step Implementation Guide

Step 1: Virtual Environment Setup

Isolated environments prevent version conflicts across Python packages:

```
# Create virtual environment
python -m venv venv

# Activate on Windows
venv\Scripts ctivate

# Activate on Linux / macOS
source venv/bin/activate
```

Step 2: Installation of Dependencies

```
# Install ChatterBot and dependencies
pip install chatterbot chatterbot-corpus pyyaml

# Download spaCy English Model
python -m spacy download en_core_web_sm
```

Step 3: Dependency File (`requirements.txt`)

```
chatterbot
chatterbot-corpus
spacy
pyyaml
```

Step 4: Custom Training Dataset (`data/custom.yml`)

Define domain-specific training pairs in YAML format to train the bot on specific questions:

```
categories:
  - conversations

conversations:
  - - Hello
    - Hi, welcome to our chatbot.
  - - What is your name?
    - I am CollegeBot, your AI assistant.
  - - Who created you?
    - I was created using Python and ChatterBot framework.
  - - What is Python?
    - Python is a high-level, interpreted programming language.
  - - What can you do?
    - I can answer basic questions, assist with college FAQs, and chat with you.
  - - Thank you
    - You are welcome!
  - - Goodbye
    - Goodbye! Have a great day ahead.
```

Step 5: Model Training Script (`train_bot.py`)

Script dedicated to reading YAML datasets and training the chatbot instance:

```
from chatterbot import ChatBot
from chatterbot.trainers import ListTrainer
import yaml

# Initialize ChatBot Instance
bot = ChatBot('CollegeBot')

# Set up Trainer
trainer = ListTrainer(bot)

# Load custom YAML training data
with open('data/custom.yml', 'r') as file:
    data = yaml.safe_load(file)

# Process conversations
for conversation in data['conversations']:
    trainer.train(conversation)

print("Training completed successfully!")
```

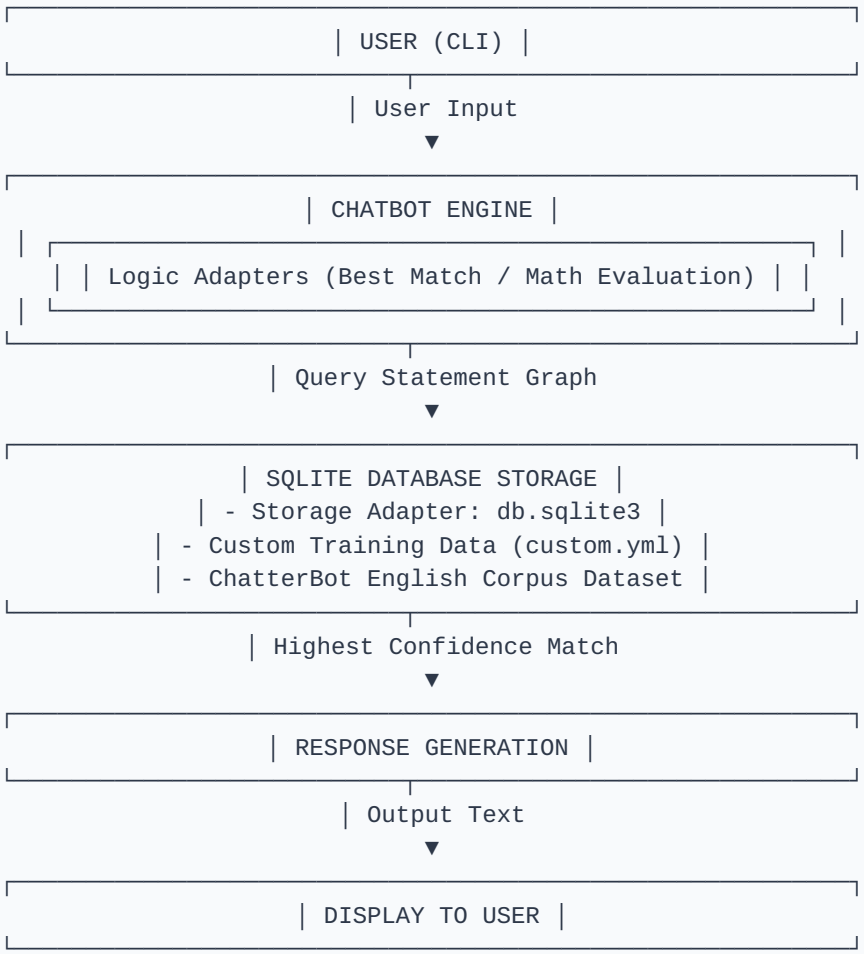
Step 6: Main Chatbot Application (`chatbot.py`)

Production-ready main script with database configuration and keyboard interrupt handling:

```
from chatterbot import ChatBot
from chatterbot.trainers import ChatterBotCorpusTrainer

# Initialize Chatbot with SQLite persistence & Best Match logic
chatbot = ChatBot(
    'CollegeBot',
    storage_adapter='chatterbot.storage.SQLStorageAdapter',
    database_uri='sqlite:///database/db.sqlite3',
    logic_adapters=[
        'chatterbot.logic.BestMatch',
        'chatterbot.logic.MathematicalEvaluation'
    ]
)

# Train with English Corpus
trainer = ChatterBotCorpusTrainer(chatbot)
trainer.train('chatterbot.corpus.english')

print("=" * 50)
print("ChatBot is ready! Type 'exit' or 'quit' to stop.")
print("=" * 50)

while True:
    try:
        user_input = input("You: ")
        if user_input.lower() in ['exit', 'quit']:
            print("Bot: Goodbye! Have a nice day.")
            break

        response = chatbot.get_response(user_input)
        print(f"Bot: {response}")

    except (KeyboardInterrupt, SystemExit):
        print("
Bot: Goodbye!")
        break
```

5. Flowchart & System Architecture

System Architecture Diagram



Execution Algorithm

1. **Initialization:** Load `ChatBot` object, logic adapters, and load connection to `db.sqlite3`.
2. **Data Ingestion:** Ingest pre-trained standard corpus and custom YAML conversation pairs into SQLite storage.
3. **Input Parsing:** Continuously monitor terminal input via an interactive execution loop.
4. **Statement Comparison:** Calculate text similarity metrics across stored statements using BestMatch.
5. **Response Generation:** Select statement response with the highest statistical confidence score.
6. **Output Display:** Print bot response to terminal prompt. Exit cleanly on terminal signal.

6. Execution Sample & Output

```
ChatBot is ready! Type 'exit' to quit.
------------------------------------------------
You: Hello
Bot: Hi, welcome to our chatbot.

You: What is your name?
Bot: I am CollegeBot, your AI assistant.

You: What is Python?
Bot: Python is a high-level, interpreted programming language.

You: How are you?
Bot: I am doing well, thank you!

You: quit
Bot: Goodbye!
```

7. Additional Extensions & Advanced Implementations

To scale this project beyond CLI and standard ChatterBot capabilities, the following enhancement scripts can be integrated:

Extension 1: Web Interface with Flask (app.py)

Transforming the chatbot into a web application using Flask REST API and frontend interface:

```
from flask import Flask, render_template, request, jsonify
from chatterbot import ChatBot

app = Flask(__name__)
chatbot = ChatBot('CollegeBot', storage_adapter='chatterbot.storage.SQLStorageAdapter')

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/get")
def get_bot_response():
    userText = request.args.get('msg')
    return str(chatbot.get_response(userText))

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

Extension 2: Modern Transformer Model Alternative (HuggingFace Integration)

For modern production applications where ChatterBot legacy dependencies may pose compatibility challenges with Python 3.11+, using HuggingFace Transformers provides modern Generative AI capabilities:

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

model_name = "microsoft/DialoGPT-medium"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)

def generate_response(user_input):
    new_user_input_ids = tokenizer.encode(user_input + tokenizer.eos_token,
return_tensors='pt')
    bot_output = model.generate(new_user_input_ids, max_length=1000,
pad_token_id=tokenizer.eos_token_id)
    response = tokenizer.decode(bot_output[:, new_user_input_ids.shape[-1]:][0],
skip_special_tokens=True)
    return response

# Example query
print("Bot:", generate_response("Hello, what is Python?"))
```

8. Project License (MIT License)

```
MIT License

Copyright (c) 2026 Rajesh

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom it is furnished to do so,
subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.
```

9. Git & GitHub Deployment Commands

```
# Initialize Git Repository
git init

# Stage all project files
git add .

# Commit files
git commit -m "Initial commit - CollegeBot ChatterBot project"

# Branch & Remote Origin setup
git branch -M main
git remote add origin https://github.com/yourusername/Chatbot-Using-ChatterBot.git

# Push code to GitHub repository
git push -u origin main
```